

Tutorial para la creación de una aplicación web con python Fast Api y docker

En este tutorial vamos a ver cómo se realiza la construcción de una aplicación web usando FastApi, el levantamiento y conexión a la base de datos en PostgreSQL, configuración y levantamiento de Docker , y un “Hola mundo” que muestre todos los aspectos mencionados anteriormente.

Para la implementación del backend se utilizó Python junto con FastAPI como framework principal para el desarrollo de APIs REST.

Librerías ORM utilizadas:

- SQLAlchemy

Configuración y levantamiento de la base de datos:

Teniendo el archivo de raise.bat o raise.sh dependiendo del sistema lo ejecutamos desde la consola:

```
./scripts/raise.bat
```

Este llamara a **docker-compose.yml** y configura el contenedor de docker y la base de datos luego entramos en un entorno de Postgresql con el nombre y la contraseña establecidas en **docker-compose.yml** luego corremos el script con la base de datos, lo mínimo necesario para el ejemplo es lo siguiente:

```

CREATE SCHEMA bienestar;

SET search_path TO bienestar;

➤ CREATE TABLE rol (
    id SERIAL PRIMARY KEY,
    nombre VARCHAR(50) NOT NULL
);

➤ CREATE TABLE usuario (
    id SERIAL PRIMARY KEY,
    nombre_completo VARCHAR(150) NOT NULL,
    correo_institucional VARCHAR(150) UNIQUE NOT NULL,
    correo_alternativo VARCHAR(150),
    password_hash VARCHAR(255) NOT NULL,
    carrera VARCHAR(100),
    creditos INT DEFAULT 0,
    meta TEXT,
    nivel_estres INT CHECK (nivel_estres BETWEEN 1 AND 5),
    activo BOOLEAN DEFAULT TRUE,
    rol_id INT,
    creado_en TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    CONSTRAINT fk_usuario_rol
        FOREIGN KEY (rol_id)
        REFERENCES rol(id)
);

```

La base de datos ya está creada en el contenedor.

Contenido archivo .yml

- docker-compose.yml

```

services:|

  db:
    image: postgres:16
    container_name: postgres_db

    restart: always

    environment:
      POSTGRES_USER: admin
      POSTGRES_PASSWORD: admin123
      POSTGRES_DB: app_db

    ports:
      - "5432:5432"

    volumes:
      - postgres_data:/var/lib/postgresql/data

volumes:
  postgres_data:

```

Hola mundo:

Se crearon una base de la app para mostrar la conexión con la base de datos en python:

Para la conexión con la base de datos tenemos un [database.py](#) de la siguiente manera:

```

from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker, declarative_base

database_url = "postgresql://admin:admin123@127.0.0.1:5432/app_db"

engine = create_engine(database_url)

SessionLocal = sessionmaker(
    autocommit=False,
    autoflush=False,
    bind=engine
)

Base = declarative_base()

```

Representación de entidades y clases minima creando un archivo [models.py](#) :

```

from sqlalchemy import Column, Integer, String
from app.database import Base

class Usuario(Base):
    __tablename__ = "usuario"
    __table_args__ = {"schema": "bienestar"}

    id = Column(Integer, primary_key=True)
    nombre_completo = Column(String)
    correo_institucional = Column(String)

```

y probamos la conexión con la base de datos creamos el endpoint en [main.py](#) :

```

from fastapi import FastAPI
from sqlalchemy.orm import Session
from app.database import SessionLocal
from app.models import Usuario

app = FastAPI()

@app.get("/hola")
def hola_mundo():

    db: Session = SessionLocal()

    usuario = db.query(Usuario).first()

    if usuario:
        return {
            "mensaje": "Hola Mundo Backend",
            "usuario": {
                "id": usuario.id,
                "nombre": usuario.nombre_completo,
                "correo": usuario.correo_institucional
            }
        }

    return {
        "mensaje": "Hola Mundo Backend",
        "usuario": None
    }

```

Se crea todo en una misma carpeta en este caso de nombre app y se luego de tener el contenedor y la base de datos funcionando, además de tener en la base de datos al menos un usuario para este ejemplo usaremos este:

```

❶ INSERT INTO bienestar.rol(nombre)
VALUES ('Estudiante');

❷ INSERT INTO bienestar.usuario(
    nombre_completo,
    correo_institucional,
    password_hash,
    rol_id
)
VALUES (
    'Pedro Castiblanco',
    'pcastiblanco@unal.edu.co',
    '123456',
    1
);

```

Ahora se ejecuta en donde se tiene la carpeta app el siguiente comando en la consola:

```
uvicorn app.main:app --reload
```

Se espera la siguiente salida:

```

op\Nacional\Ingesoft\MODS-TLAUNCHER-project-backend>python -m uvicorn app.main:app --reload
Will watch for changes in these directories: ['C:\\Users\\RPC\\Desktop\\Nacional\\Ingesoft\\MODS-TLAUNCHER-project-backend']
Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
Started reloader process [36m16324[0m using [36m1mStatReload[0m
Started server process [36m19400[0m
Waiting for application startup.
Application startup complete.

```

Ahora se pone en el navegador <http://127.0.0.1:8000/hola> y se obtiene este resultado:

← → ↻ ⓘ http://127.0.0.1:8000/hola

Dar formato al texto ☒

```
{
  "mensaje": "Hola Mundo Backend",
  "usuario": {
    "id": 1,
    "nombre": "Pedro Castiblanco",
    "correo": "pcastblanco@unal.edu.co"
  }
}
```